

Python Coding for AI

Ch1: Python의 직관적 이해

Ch2: Python OOP

Ch3: What is Numpy & Why Numpy?

Ch4: What is Pandas & Why Pandas?

Ch5: What is Matplotlib & Why Matplotlib?

Ch6: What is Sklearn & Why Sklearn?

Ch7: What is Data Preprocessing & Why Data Preprocessing?

강사: 김 형 주 교수



■ 연구분야

- 빅데이터 처리 기술 (Big Data Processing Technology)
- Python Coding for AI
- AI 기업교육

■ 학력

- 학사 (1982): 서울대학교 컴퓨터공학
- 석사 (1985): 미국 Univ of Texas at Austin 전산학 (Computer Science)
- 박사 (1988): 미국 Univ of Texas at Austin 전산학 (Computer Science)

■ 경력

- 1988-- 1990: 미국 조지아 공과대학 (Georgia-Tech) Computer Science 학과, 조교수
- 2003 -- 2004: 서울대학교 정보화본부 본부장 (서울대 CIO) & 대한민국교육망 (KREN) 운영본부장
- 2007 -- 2010: 서울대 컴퓨터공학부 학부장 (Head, School of Comp Sci & Eng)
- 2012 – 2014: SNU Big Data Center 센터장
- 2010 – 2014: 재단법인 서울대 발전기금 상임이사
- 2018 -- 2023: 서울대-삼성 DS²과정 주임교수
- 1991 -- **현재**: 서울대 컴퓨터공학부 조교수, 부교수, 교수, **명예교수**
- 2025 – **현재**: 서울공대 AI기업교육센터 **센터장**
- 2012 -- **현재**: 한국공학한림원 정회원 (NAEK Fellow), **원로회원**
- 2012 -- **현재**: 학국과학기술한림원 정회원 (KAST Fellow), **학술부원장**

기업의 AI전환은 서울대가 이끁니다

홈페이지: sacta.snu.ac.kr

서울대학교 AI 기업교육 센터 SNU AI Corporate Training

원훈철 고금 시 기업 교육위원의 지적·배경·대응

- 하이브리드 수업구조: 70% 야간 줌수업, 30% 대면수업
- 다양한 AI교육과정들과 기업맞춤형 교육
- 서울대 AI대학원 교수 73명 출강

서울대학교 AI 기업교육 센터

08826 서울시 관악구 관악로 1호동
(해동첨단공학관) 6층 662호

AI 경쟁력은 서울대 sacta와 함께

Event

한림원탁토론회 'AI 프런티어

한국과학기술한림원이 오는 9월 10일부터 11월까지 'S&T x AI: 다
STEM 교육, K-방산, 신약개발, 신소재, 뇌-컴퓨터 인터페이스 등



13주 우리금융그룹-서울대 AI/데이터 전문가과정

2026.08.07(금) Python for AI 김형주 교수

2026.08.14(금) Data Visualization 서진욱 교수

2026.08.21/28/9.04(금) Machine Learning 김용대 교수

2026.09.11(금) AI ethics and regulation 천현득 교수, 박상철 교수

2026.09.18/10.02/16/23(금) Deep learning 김건희 교수

2026.10.30(금) DB for AI 이상구 교수

2026.11.06/13(금) LLM & FAG 서봉원 교수

2026.11.20(금) Reinforcement learning 원정담 교수

2026.11.27(금) Computer Vision 최종현 교수

2026.12.04(금) 금융 AI산업 및 AI Service 강유, 박우진, 장우진 교수

2026.12.18(금) AI트랜스포메이션 유병준 교수

Python1

vs.

Python2

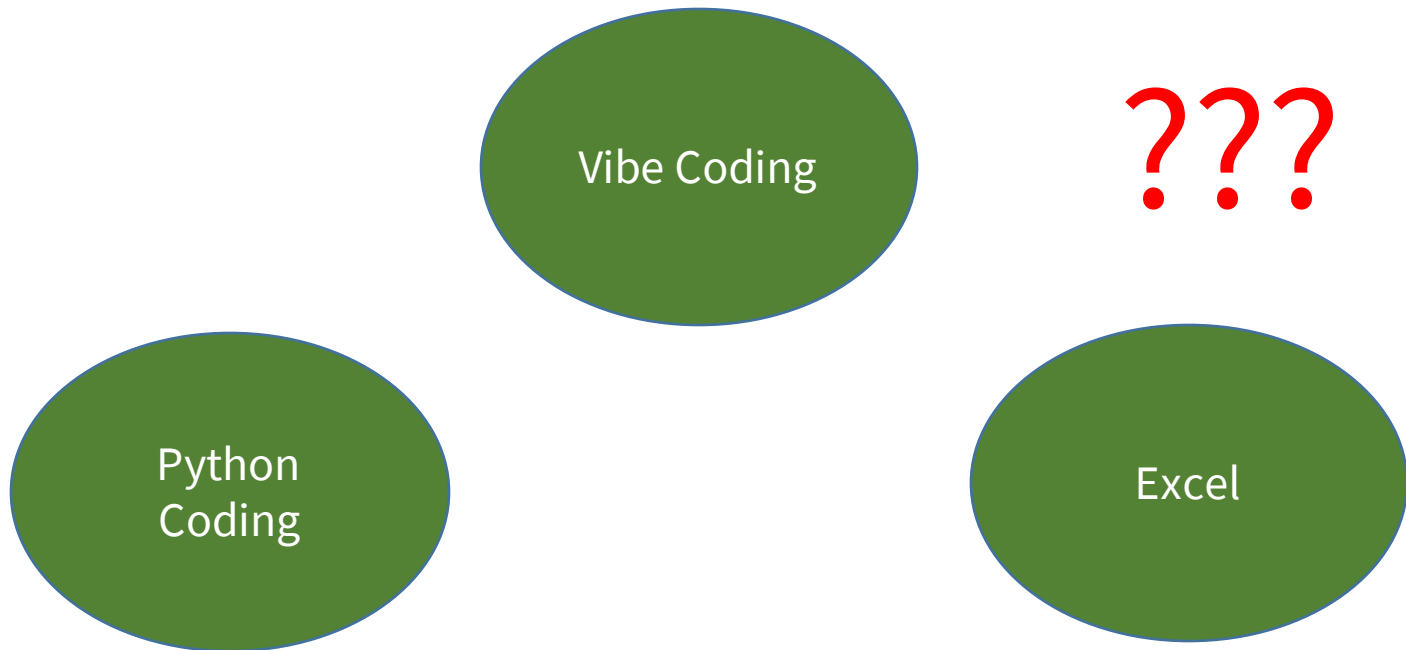
Python-1 커리큘럼 (42 시간)



Python-2 커리큘럼 (42시간)

- Basic Data Types
- Collection Data Types
- Python 기본 문법과 Function의 작성
- Built-in Functions
- File IO
- Exception Handling
- Module, Standard Library
- Recursion 함수 작성
- Object-Oriented Programming

- Numpy
- Pandas
- Matplotlib & Seaborn
- Scipy
- SK-learn
- Data Preprocessing



Question1: 딥러닝 기법들이 전통 AI기법들을 압도하는데 왜 전통 AI기법들을 배워야 하지?

- 딥러닝이 이미지·텍스트·음성 같은 비정형 데이터에서 압도적
- 그러나 전통 AI기법을 써야 하는 이유!
 - 데이터가 적거나 정형(tabular) 데이터에서 충분한 성능 발휘
 - 연산 비용이 훨씬 가볍다
 - 해석 가능성(설명력)이 필요한 금융·의료·법률 분야에선 블랙박스인 딥러닝 회피
 - 과적합·정규화·손실함수 같은 전통 AI기법 핵심 개념의 기초없이 딥러닝도 없다!

Question2: 생성형 AI기법들이 딥러닝 기법(CNN, RNN)들을 압도하는데 왜 딥러닝 기법들을 배워야 하지?

- 생성형 AI(대부분 Transformer 기반)가 언어·이미지 생성에서 압도적
- 그러나 기존의 딥러닝을 배워야 하는 이유
 - 생성형 AI의 Transformer의 attention 원리는 CNN과 RNN의 구조를 모르면 이해불가
 - 역전파·손실함수·과적합같은 딥러닝의 핵심 학습 원리
 - 정형화된 비전 태스크·엣지 환경에서는 CNN이 여전히 더 가볍고 정확
 - 정형화된 비전 태스크: 입력 이미지의 종류와 나올 수 있는 답이 미리 정해져 있는 좁고 명확한 작업
 - 엣지 환경: 클라우드 서버가 아니라 현장의 로컬 기기 — 자동차 내장 컴퓨터, 공장 카메라, 드론, 병원 내부 PC
 - 자율주행 인식, 공장 불량 검출, 의료영상 판독 등엔 지금도 CNN모델이 실무 표준
 - 좁고 명확한 태스크·소량 데이터·저비용 환경에서는 기존 딥러닝이 저렴*정확

Question3: Excel로도 데이터분석이 되는데 왜 Python?

- Excel로 충분한 경우
 - 기초 통계 처리에 아주 훌륭
 - 회귀분석, 이상치 탐지, K-means clustering 정도만 한다면 OK
 - 데이터가 작고(몇만개 이하) 깨끗하면 OK
- AI Machine Learning기술들로 확장되면 Python으로 해결해야 함
 - 의사결정나무, 랜덤포레스트, 로지스틱회귀, KNN
 - SVM, Naïve Bayes, PCA, Q-Learning
 - 하이퍼파라미터 튜닝
 - CNN 딥러닝, RNN 딥러닝, Transformer 딥러닝

Question4: Vibe Coding으로 Python Coding을 대체하는데 왜 Python을 배워야 하지?

- Vibe Coding을 통해 AI가 생성하는 코드는 결국 Python 코드!!!
- Python의 기본 문법과 AI Coding관련 Module (Numpy, Pandas, Sklearn 등등)를 모르면, Vibe coding으로 만들어준 코드를 읽고 이해할 수 없다!
- Vibe Coding을 배우는데도 Python Coding을 배우는 이유
 - Vibe Coding AI를 정확히 지시·활용하기 위한 최소한의 Python 독해력 필요
 - Vibe Coding이 생성한 Python Code와 결과가 틀렸을 때 (할루시네이션, 논리 오류) 잡아낼 수 있어야 한다

Enterprise Python Coding for AI

- **Vision:** 현업 Non-IT 엔지니어들이 실제 AI 프로젝트나 AX(AI Transformation) 업무 수행 가능한 핵심 실무 역량을 보유
- **기본: 데이터 분석(Scikit-learn) AI Coding**
- **딥러닝의 PyTorch 기반 파이프라인 Coding**
- **'생성형 AI 기반 앱 개발 (OpenAI/Gemini API, RAG, Function Calling)'의 기본 코딩 구조를 이해**

목표: PyTorch를 활용하여 딥러닝의 원리를 코드로 구현하고, Tensor 연산 및 모델 학습/평가 루프를 습득

1. 기본 데이터 구조 및 자동 미분 (Tensor & Autograd)

- **Tensor 및 GPU 할당:** NumPy 배열-Tensor 전환 및 `cuda()/device`를 활용한 GPU 메모리 할당
- **자동 미분 (Autograd):** `requires_grad=True`로 변수를 지정하고, 실시간 계산 경로를 기록해 역전파(가중치 갱신) 수행

2. 신경망 아키텍처 구축 (torch.nn)

- **기본 모듈 설계:** `nn.Module` 상속을 통한 커스텀 신경망 구현
 - 선형: `nn.Linear`
 - 합성곱: `nn.Conv1d`, `nn.Conv2d`, `nn.Conv3d` (특징 추출)
 - 순환: `nn.RNN`, `nn.LSTM`, `nn.GRU` (시계열/자연어)
 - 트랜스포머: `nn.Transformer`, `nn.MultiheadAttention`
 - 풀링/정규화: `nn.MaxPool2d`, `nn.BatchNorm2d`, `nn.LayerNorm`
- 활성화 함수: `nn.ReLU`, `nn.GELU`, `nn.Sigmoid`, `nn.Softmax`

3. 데이터 파이프라인 및 표준 학습 루프 (Training Loop)

- **데이터 로딩:** `Dataset` 및 `DataLoader`를 활용한 Batching, Shuffling 구현
- 손실함수 및 최적화: `nn.CrossEntropyLoss`, `nn.MSELoss` 및 `torch.optim` (Adam, SGD)
- 표준 학습 4단계: Forward → Loss 계산 → Backward (역전파) → Optimizer Step (가중치 갱신)

4. 실전 응용 및 과적합 방지 (Vision / NLP Baseline)

- 도메인 모델 활용: `torchvision.models` (ResNet 등) 사전학습 모델 및 전처리 활용
- 성능 최적화 기법: `nn.Dropout`, 조기 종료(Early Stopping), 학습률 스케줄러 코딩

목표: OpenAI, Gemini 등 최신 LLM API를 파이썬으로 제어하고, RAG 및 도구 연동(Function Calling)을 활용해 기업형 AI 애플리케이션 구축하기

1. LLM API 연동 및 프롬프트 제어

- **API 연결 및 보안:** OpenAI / Google Gemini SDK 설정 및 API Key 안전한 관리
- **대화 구조 설계:** System / User / Assistant 메시지 역할을 명확히 분리하여 대화 맥락 유지
- **답변 제어:** Temperature, Top-p 등으로 답변의 창의성·정확도를 조절하고, 실시간 응답(Streaming) 처리

2. 정형화된 출력 및 외부 도구 연동 (Tool Use)

- **규격화된 응답 (Pydantic):** AI의 답변을 정해진 형식(JSON Schema)으로 고정하여 시스템 에러 방지
- **외부 기능 호출 (Function Calling):** AI가 사내 파이썬 함수나 외부 API를 직접 불러와 실행하도록 연결
- **에이전트 기초 (ReAct):** AI가 스스로 판단하고 도구를 골라 문제를 해결하는 에이전트(Agent) 패턴 코딩

3. 사내 데이터 기반 답변 검색 엔진 (RAG Engine)

- **문서의 숫자화 (Embedding):** 텍스트(문장/문서)를 AI가 이해할 수 있는 의미 단위의 숫자(벡터)로 변환
- **벡터 DB 연동:** ChromaDB / FAISS를 활용해 질문과 가장 유사한 사내 문서 검색
- **사내 문서 기반 RAG:** PDF, TXT 등 사내 문서를 바탕으로 환각(Hallucination) 없는 정확한 답변을 생성하는 풀 스택 파이프라인 구축

4. 고급 AI 프레임워크 및 웹 서비스 결합

- **LLM 오케스트레이션:** LangChain / LlamaIndex의 핵심 요소(Chains, Memory, Retrievers) 코딩
- **웹 서비스화 (FastAPI):** 구축한 AI 엔진을 REST API 웹 프레임워크와 결합하여 실제 서비스로 배포하기

자! ... 이제!!

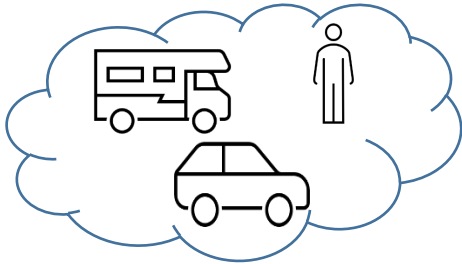
* 마음을 편안하게 가지고 Python Review를 합시다!

*12주 과정의 강의진행중에도 틈을 내서 Python AI Coding을 연마하면 더욱 유익한 수강이 될것입니다

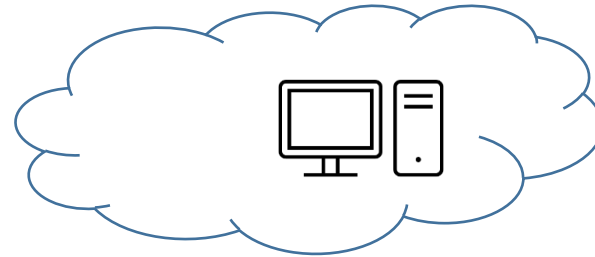
Let's Go!

Programming 이란 무엇인가?

실세계의 문제



컴퓨팅세계의 문제



실세계의 문제

➔ 데이터로 표현

➔ 코딩 (Programming)

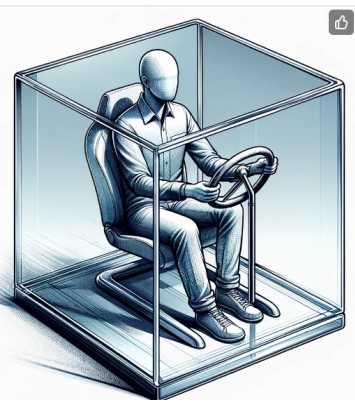
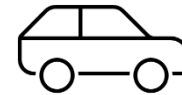
- Built-In Data Types
- User-defined Data Types

- Data Structure
- Algorithm

Programming 예제

Real-World Problem: 자동차의 운전자의 사용가능 공간의 크기를 계산하라!

=> 임의의 차량에 대하여 차량내부의 가로, 세로, 높이를 안다면 사용 가능 공간을 구할 수 있음.



User 사용공간
계산에 필요한
데이터:

가로
세로
높이

문제의 표현
(Representation)

x: 가로
y: 세로
z: 높이

Coding

```
def volume_compute(x, y, z):  
    return x * y * z
```


초심자가 Python Coding을 적절하게 하려면!

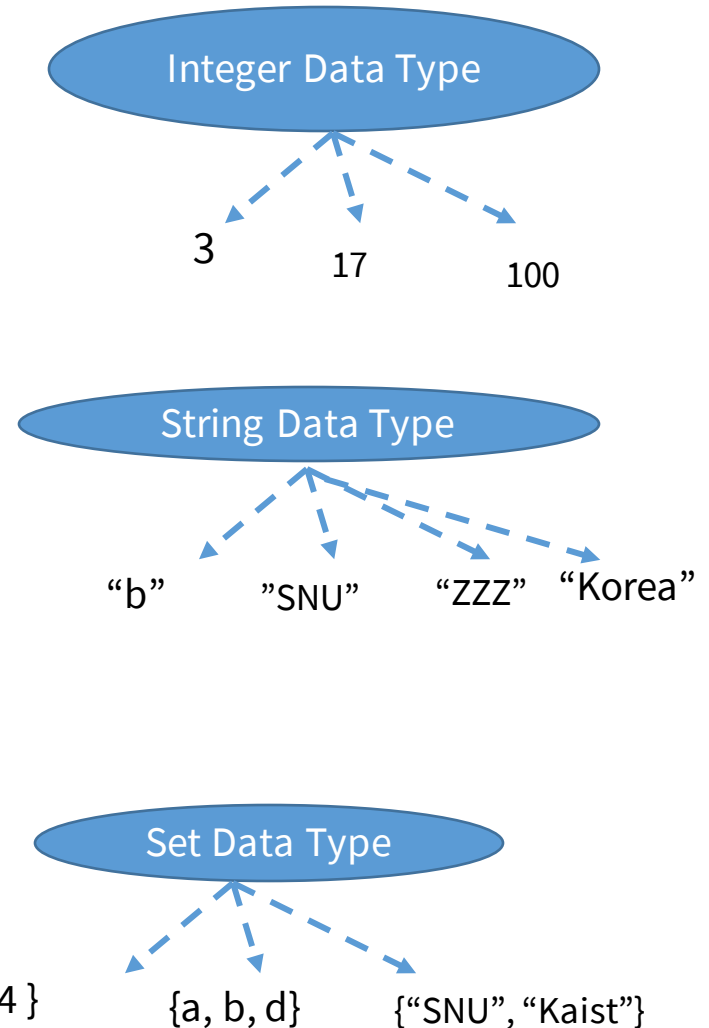
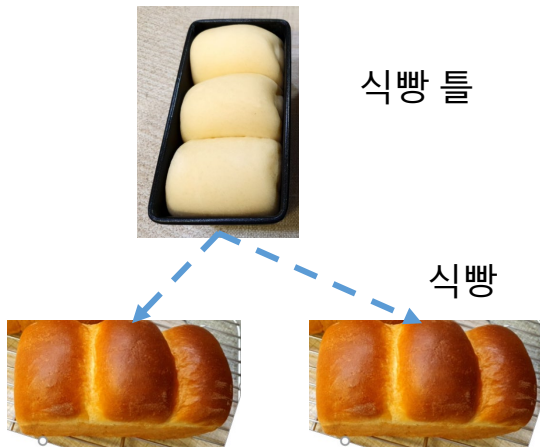
- Python **Built-in Data Type**들의 이해와 사용
 - Python이 미리 구현한 Class(=Data Type)들
- Python **Built-in Function**들의 이해와 사용
 - Python이 미리 구현한 Function들
- Python **Reserved Word (Keyword)**들의 이해와 사용
 - Python이 이 자기만 사용하려고 지정한 단어들
- Python **Standard Library**들의 이해와 사용
 - Python 이 미리 구현한 Library들
- Python Language의 **기본 Statement (Grammar)**들의 이해와 사용
 - Assignment (=) Statement
 - IF Statement
 - FOR Statement

우리주변에 있는 여러 형태의 데이터

- 정수들 : 2, 7, -29
- 실수들: 0.4, -23.45, 198.64
- 논리값: True, False
- 문자열: “서울대”, “양의지”
- 순서없는 데이터의 집합(set):
 - { “청주대”, “성균관대”, “울산대”, }
 - { 3, 14, 9, 10, 7, 8 }
- 순서있는 데이터의 리스트(list):
 - [“Jan“, “Feb“, “Mar“, “Apr“, ..., “Nov“, “Dec“]
 - [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

사물의 틀(형) vs.

Data Type (데이터 타입)



Python Built-in Data Types

Basic Data Types vs. Collection Data Types (기본 데이터 타입) (모음 자료형)

- **Int (정수)**

X = 3

- **Float (실수)**

X = 3.1

- **Boolean (논리값)**

X = True

Y = False

- **String (문자열)**

X = "CSM"

- **List (리스트)**

X = [3, 7, 2, 1]

- **Tuple (튜플)**

X = (3, 7, 2, 1)

- **Set (집합)**

X = {3, 7, 2, 1}

- **Dictionary (딕셔너리)**

X = {"Korea": 3500, "USA": 2000}

프로그래밍언어를 배우는 첫걸음은 그 언어에서 제공되는 데이터 타입을 잘 이해하고 사용법을 배우는것이다!!!

Built-In Data Type의 이해하고 사용

- $s1 = \{'a', 'a', 'b', 'c', 'c', 'c'\}$, $s2 = \{'b', 'd', 'e'\}$ 일때 $s1 - s2$ 의 값은? $s1 \mid s2$ 의 값은?
- $lst1 = [2, 7, \text{"kim"}]$, $lst2 = [2, 9, \text{"lee"}]$ 일때 $lst1 + lst2$ 의 값은?
- List data type에 속한 함수들은 어떤것들이 있는가?
- String data type에 속한 함수들은 어떤것들이 있는가?
- List data type과 String data type에서 data mutability는 어떻게 되는가?
- List data type과 Tuple data type의 차이점은어떻게 되는가?
- Set data type이 가지는 성질들은 어떤것들이 있는가?
- Dictionary data type은 set data type과 어떻게 다른가?

Python Built-in Functions

Python 3.13 Built-in Functions (71개, 2025년)

Built-in Functions

A

`abs()`
`aiter()`
`all()`
`any()`
`anext()`
`ascii()`

B

`bin()`
`bool()`
`breakpoint()`
`bytearray()`
`bytes()`

C

`callable()`
`chr()`
`classmethod()`
`compile()`
`complex()`

D

`delattr()`
`dict()`
`dir()`
`divmod()`

E

`enumerate()`
`eval()`
`exec()`

F

`filter()`
`float()`
`format()`
`frozenset()`

G

`getattr()`
`globals()`

H

`hasattr()`
`hash()`
`help()`
`hex()`

I

`id()`
`input()`
`int()`
`isinstance()`
`issubclass()`
`iter()`

L

`len()`
`list()`
`locals()`

M

`map()`
`max()`
`memoryview()`
`min()`

N

`next()`

O

`object()`
`oct()`
`open()`
`ord()`

P

`pow()`
`print()`
`property()`

R

`range()`
`repr()`
`reversed()`
`round()`

S

`set()`
`setattr()`
`slice()`
`sorted()`
`staticmethod()`
`str()`
`sum()`
`super()`

T

`tuple()`
`type()`

V

`vars()`

Z

`zip()`

`__import__()`

Built-in 함수를 이해하고 사용

```
fruits = ["apple", "banana", "cherry", "date", "elderberry"]
```

```
# fruits의 단어중에 사전순으로 정렬시
```

```
# 가장 마지막에 나오는 단어를 찾아라
```

```
longest_fruit = max(fruits)
```

```
print("The longest fruit name is:", longest_fruit)
```

```
# fruits의 문자열들을 알파벳 순서에 따라 정렬 하라
```

```
sorted_fruits = sorted(fruits)
```

```
print("Fruits sorted by length:", sorted_fruits)
```

```
numbers = [2, -3, 5, -8, 10]
```

```
# numbers의 평균값을 계산하라
```

```
average = sum(numbers) / len(numbers)
```

```
print("Average of absolute values:", average)
```

```
# 평균값을 소수 3번째자리에서 반올림하라
```

```
rounded_average = round(average, 2)
```

```
print("Rounded average:", rounded_average)
```

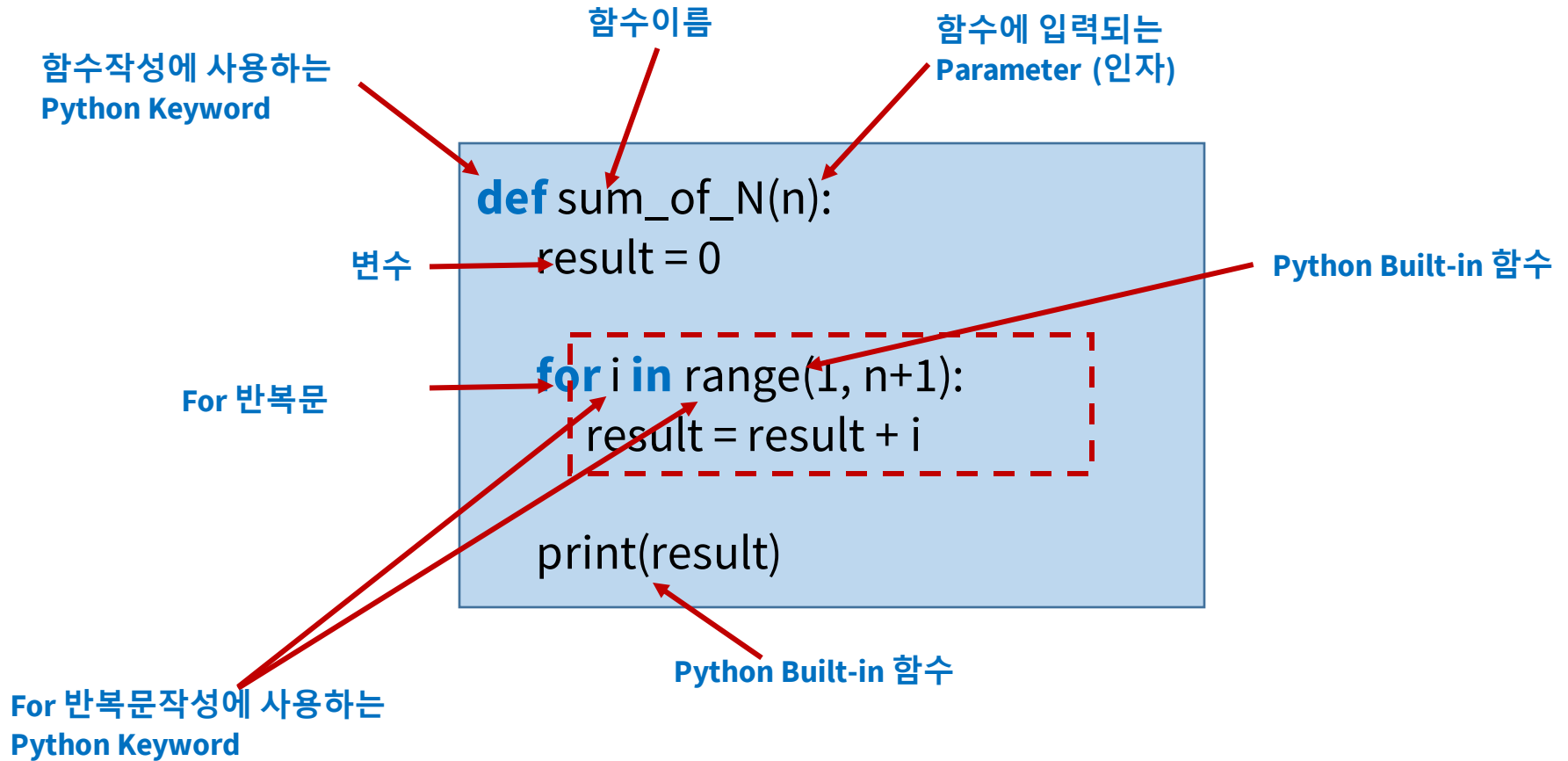
Python Reserved Words (Keywords)

- 변수, 함수, 클래스의 이름으로 예약어(=키워드)를 사용하는 것은 금지.

Python 3.13 Reserved Word (35개, 2025년)

False	True	None	await	and
as	assert	async	class	continue
def	del	else	elif	except
finally	for	from	import	pass
break	if	in	is	lambda
global	or	raise	return	try
nonlocal	while	not	with	yield

Python 함수작성의 구성 요소



Standard Library를 이해하고 사용

- 250여개
- 수학의 많은 함수를 보유 : math
- 통계관련 많은 함수를 보유: statistics
- 랜덤숫자관련 많은 함수를 보유: random
- ...

```
>>> import math
```

```
>>> math.factorial(7)
5040
```

```
>>>
```

```
>>> math.factorial(7.4)
Traceback (most recent call last):
```

```
>>> math.gcd( 6, 9)
3
```

```
>>>
```

```
>>> math.gcd( 60, 15)
15
```

```
>>>
```

```
>>> math.gcd( 6.0, 9.0)
Traceback (most recent call last):
```

```
>>> import statistics
```

```
>>> s_data = [1.5, 2.5, 2.5, 2.75, 3.25, 4.75]
```

```
>>> statistics.stdev(s_data)
1.0810874155219827
```

```
>>> statistics.variance(s_data)
1.16875
```

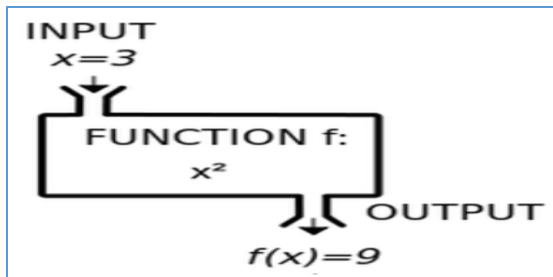
초심자가 Python Coding을 적절하게 하려면!

- Python **Built-in Data Type**들의 이해와 사용
 - Python이 미리 구현한 Class(=Data Type)들
- Python **Built-in Function**들의 이해와 사용
 - Python이 미리 구현한 Function들
- Python **Reserved Word (Keyword)**들의 이해와 사용
 - Python이 이 자기만 사용하려고 지정한 단어들
- Python **Standard Library**들의 이해와 사용
 - Python 이 미리 구현한 Library들
- Python Language의 **기본 Statement (Grammar)**들의 이해와 사용
 - Assignment (=) Statement
 - IF Statement
 - FOR Statement

Python Coding을 다시 정리한다면....

- Python으로 특정문제를 해결하기 위해서 앞에서 제공된 것들 (**Built-in Data types, Built-in Functions, Keywords, Standard Library**)을 사용하여 필요한 **function & class 구현**

- **Function 구현**: 초중고에서 배웠던 함수 $f(x)$ 구현

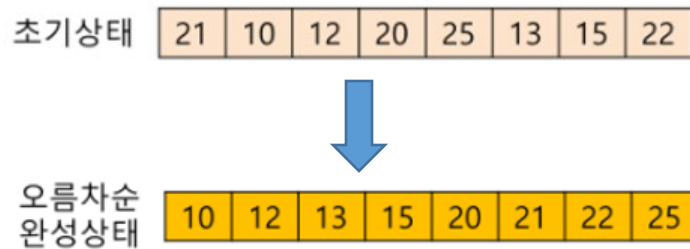


- **Class (= Data Type) 구현**: Data의 틀을 만드는것

Student Class: 학생 Data틀
Fruit Class: 과일 Data틀
...

Built-in Data Type 만을 가지고 Function을 구현하는 예 [1/2]

** 숫자의 정렬 (Sorting)



** 숫자의 위치 찾기 (Searching)

- 아래 숫자중에서 6의 위치를 찾아라!
- 780, 600, 999, 431, 80, 590, 6, 985, 578, 279

Built-in Data Type 만을 가지고 Function을 구현하는 예 [2/2]

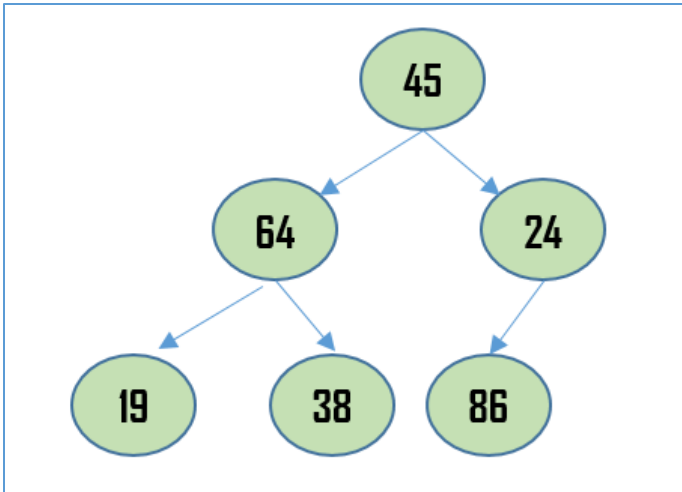
** 2~100까지의 소수를 구하라:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97

** π 값을 추정하라!

- $\pi \approx 3.14159265358979323846\dots\dots$
- 수백년간 수많은 수학자들이 π 값을 계산하기 위한 다양한 방법을 연구

Tree 구조에 데이터를 저장하고 싶다면 [1/2]



- Insert a new node
- Delete an existing node
- Search a location of a particular value
- Find min/max among values in the tree

• ...

List로 구현

[45, 64, 24, 19, 38, None, 86]

Level 1 Level 2 Level 3

Nested List 구현

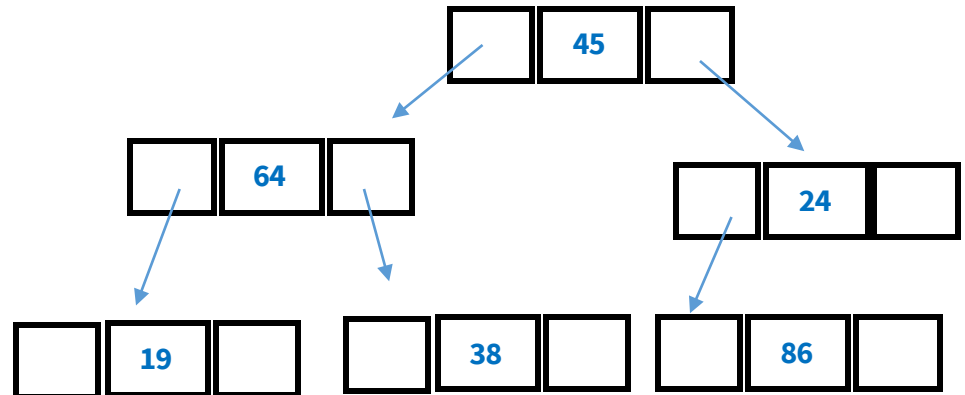
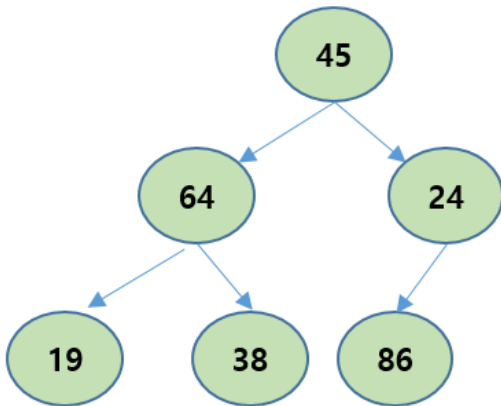
- [45, left, right]
- [45, [64, left, right], [24, left, []]
- [45, [64, [19, [], []], [38, [], []]], [24, [86, [], []], []]

Tree 구조에 데이터를 저장하고 싶다면 [2/2]

Tree Structure를 위한 Data Type이 있으면 좋겠다!

User-defined Type (class) → BinaryTreeNode Class

Left Pointer	Node Value	Right Pointer
-----------------	---------------	------------------



Built-in Data Type vs. User-defined Data Type

- **Python Basic Data Types (기본 데이터 타입)**

- **int**
- **float**
- **Bool**

- **Python Collection Data Types**

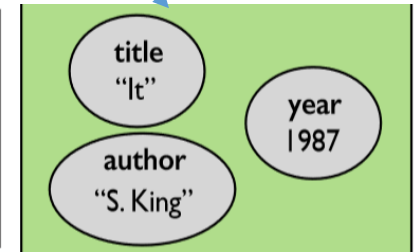
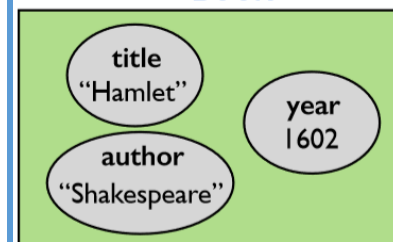
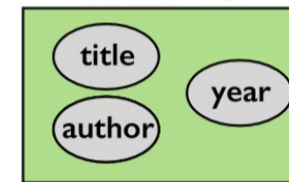
(컨테이너 데이터 타입)

- **str**
- **list**
- **tuple**
- **set**
- **dict**

Books in a library



Book class



Python Object Oriented Programming (OOP)